

Lab 12

Spring Boot login/Logout (Spring Security + H2 + Thymeleaf)

In this lab we shall implement "login/logout" and "authentication/authorization" in a "Spring Boot 3" application.

To demonstrate the functionality, a new "Spring Boot" application is created with the help of spring initializr.

Meet the Spring team this August at SpringOne.



Project
☐ Gradle - Groovy
☐ Gradle - Kotlin
☒ Maven

Language
☒ Java ☐ Kotlin
☐ Groovy

Spring Boot
☐ 3.1.1 (SNAPSHOT) ☒ 3.1.0 ☐ 3.0.8 (SNAPSHOT)
☐ 3.0.7 ☐ 2.7.13 (SNAPSHOT) ☐ 2.7.12

Project Metadata

Group

Artifact

Name

Description

Package name

Packaging ☒ Jar ☐ War

Java ☐ 20 ☒ 17 ☐ 11 ☐ 8

Dependencies

Spring Boot Actuator **OPS**

Supports built in (or custom) endpoints that let you monitor and manage your application - such as application health, metrics, sessions, etc.

H2 Database **SQL**

Provides a fast in-memory database that supports JDBC API and R2DBC access, with a small (2mb) footprint. Supports embedded and server modes as well as a browser based console application.

Spring Data JPA **SQL**

Persist data in SQL stores with Java Persistence API using Spring Data and Hibernate.

Spring Security **SECURITY**

Highly customizable authentication and access-control framework for Spring applications.

Thymeleaf **TEMPLATE ENGINES**

A modern server-side Java template engine for both web and standalone environments. Allows HTML to be correctly displayed in browsers and as static prototypes.

Validation **I/O**

Bean Validation with Hibernate validator.

Spring Web **WEB**

Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

In order to save/get user details (name, email or username, roles) the application shall use the H2 database. The application also uses Thymeleaf as a server-side template engine. We shall create a custom "registration" and "login" flow based on "thymeleaf (html)" forms.

Dependencies

pom.xml

The "spring-boot-starter-security" combines Spring Security dependencies, and the "spring-boot-starter-web" enables "Tomcat" as a default embedded server.

Like wise, "spring-boot-starter-data-jpa" provides the "data-jpa" dependency to efficiently connect Spring applications with relational databases.

Below shows required dependencies:

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-thymeleaf</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>org.thymeleaf.extras</groupId>
    <artifactId>thymeleaf-extras-springsecurity6</artifactId>
  </dependency>

  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>org.springframework.security</groupId>
    <artifactId>spring-security-test</artifactId>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
    <scope>runtime</scope>
  </dependency>
</dependencies>
```

Model Package

Create two classes named as “User” and “Role” and annotate as entities (@Entity) in a package called “model”.

Many-to-many relationship between User and Role.

```
8  @Entity
9  @Table(name = "users")
10 public class User {
11     @Id
12     @GeneratedValue(strategy = GenerationType.IDENTITY)
13     private Long id;
14
15     @Column(nullable = false)
16     private String name;
17
18     @Column(nullable = false, unique = true)
19     private String email;
20
21     @Column(nullable = false)
22     private String password;
23
24     @ManyToMany(fetch = FetchType.EAGER, cascade = CascadeType.ALL)
25     @JoinTable(
26         name = "users_roles",
27         joinColumns = {@JoinColumn(name = "user_id", referencedColumnName = "id")},
28         inverseJoinColumns = {@JoinColumn(name = "role_id", referencedColumnName = "id")})
29     )
30     private List<Role> roles = new ArrayList<>();
31
32     public User() {
33     }
34
35     public User(String name, String email, String password, List<Role> roles) {
36         this.name = name;
37         this.email = email;
38         this.password = password;
39         this.roles = roles;
40     }
41 }
```

Source path: src/main/java/com/csc3402/security/formlogin/model/User.java

Note: Remember to create

1. Getters & Setters,
2. All-arguments & non-arguments constructor,
3. toString

The "User" and "Role" are plain old models representing a Spring Security user and associated role in the application and database.

```

7
8     @Entity
9     @Table(name = "roles")
10    public class Role {
11
12        @Id
13        @GeneratedValue(strategy = GenerationType.IDENTITY)
14        private Long id;
15
16        @Column(nullable = false, unique = true)
17        private String name;
18
19        @ManyToMany(mappedBy = "roles")
20        private List<User> users = new ArrayList<>();
21
22        public Role(String name) { this.name = name; }
23
24
25        public Role() {
26        }
27
28
29        public Role(String name, List<User> users) {
30            this.name = name;
31            this.users = users;
32        }

```

source path: src/main/java/com/csc3402/security/formlogin/model/Role.java

Note: Remember to create

1. Getters & Setters,
2. All-arguments & non-arguments constructor,
3. toString

The "Role" class has a list of users, which means a user can have multiple roles, and a role can be associated with a number of users.

DTO Package

DAO (Data Access Object) and DTO (Data Transfer Object) patterns are used in Object Relational Mapping. DAO acts as a bridge between the database and the application. DTO acts as a data store from where the data is received and transferred to different layers i.e., to the DAO application.

Create a package called 'dto' and a class called "UserDto"

```
import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotEmpty;

public class UserDto {

    private Long id;

    @NotEmpty(message = "Please enter valid name.")
    private String name;

    @NotEmpty(message = "Please enter valid email.")
    @Email
    private String email;

    @NotEmpty(message = "Please enter valid password.")
    private String password;

    public UserDto() {
    }

    public UserDto(String name, String email, String password) {
        this.name = name;
        this.email = email;
        this.password = password;
    }
}
```

Source path: src/main/java/com/csc3402/security/formlogin/dto/UserDto.java

Note: Remember to create

1. Getters & Setters,
2. All-arguments & non-arguments constructor,
3. toString
- 4.

The "UserDto" class is a helper class to be used in "Html form to Controller communication" when a new user is registered in the system.

Repository Package

Create two repositories called "UserRepository" and "RoleRepository" in repository package. The "UserRepository" and "RoleRepository" are using **Spring Data JPA** to significantly improve the implementation of data access layers by reducing the effort to the amount that's actually needed.

```
1 package com.csc3402.security.formlogin.repository;
2
3 import com.csc3402.security.formlogin.model.User;
4 import org.springframework.data.jpa.repository.JpaRepository;
5 import org.springframework.stereotype.Repository;
6
7 @Repository
8 public interface UserRepository extends JpaRepository<User, Long> {
9     User findByEmail(String email);
10 }
11
```

Source path: src/main/java/com/csc3402/security/formlogin/repository/UserRepository.java

The "findByName" method gets the "Role" object for the associated "role" name.

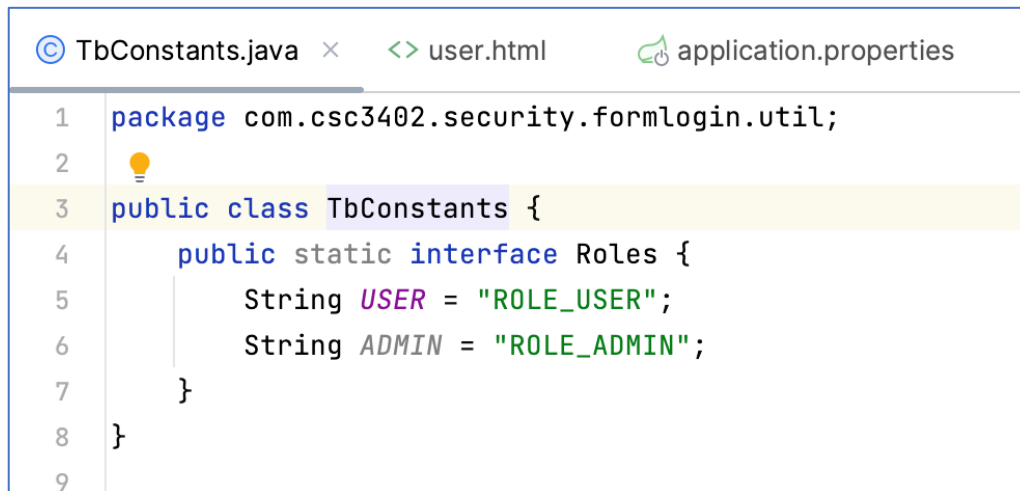
```
1 package com.csc3402.security.formlogin.repository;
2
3 import com.csc3402.security.formlogin.model.Role;
4 import org.springframework.data.jpa.repository.JpaRepository;
5 import org.springframework.stereotype.Repository;
6
7 @Repository
8 public interface RoleRepository extends JpaRepository<Role, Long> {
9     Role findByName(String name);
10 }
11
```

Source path: src/main/java/com/csc3402/security/formlogin/repository/RoleRepository.java

Constants (util package)

The "TbConstants.java" maintains constants used throughout the application; this helps make the application code more maintainable.

Create a package called "util" and a java class called "TbConstants" as follows :-



```
TbConstants.java x user.html application.properties
1 package com.csc3402.security.formlogin.util;
2
3 public class TbConstants {
4     public static interface Roles {
5         String USER = "ROLE_USER";
6         String ADMIN = "ROLE_ADMIN";
7     }
8 }
9
```

Service Package

Next, create a package call "service" and an interface and class as follows:-

1. UserService.java - Interface
2. UserServiceImpl.java - class

"UserService.java" and "UserServiceImpl.java" interface represent an intermediate service layer between data/repository and controller.



```
UserService.java x
1 package com.csc3402.security.formlogin.service;
2
3 import com.csc3402.security.formlogin.dto.UserDto;
4 import com.csc3402.security.formlogin.model.User;
5
6 public interface UserService {
7
8     void saveUser(UserDto userDto);
9     User findUserByEmail(String email);
10 }
11
```

Source path: src/main/java/com/csc3402/security/formlogin/service/UserService.java

The "saveUser()" method converts "UserDto" to "User" and saves it in the database (H2).
Note: UserServiceImpl is annotated with **@Service** annotation

```
14
15 @Service
16 public class UserServiceImpl implements UserService{
17     @Autowired
18     private UserRepository userRepository;
19
20     @Autowired
21     private RoleRepository roleRepository;
22
23     @Autowired
24     private PasswordEncoder passwordEncoder;
25     @Override
26
27     public void saveUser(UserDto userDto) {
28         Role role = roleRepository.findByName(TbConstants.Roles.USER);
29
30         if (role == null)
31             role = roleRepository.save(new Role(TbConstants.Roles.USER));
32
33         User user = new User(userDto.getName(), userDto.getEmail(), passwordEncoder.encode(userDto.getPassword()),
34             Arrays.asList(role));
35         userRepository.save(user);
36     }
37
38     @Override
39     public User findUserByEmail(String email) { return userRepository.findByEmail(email); }
40
41 }
42
43
```

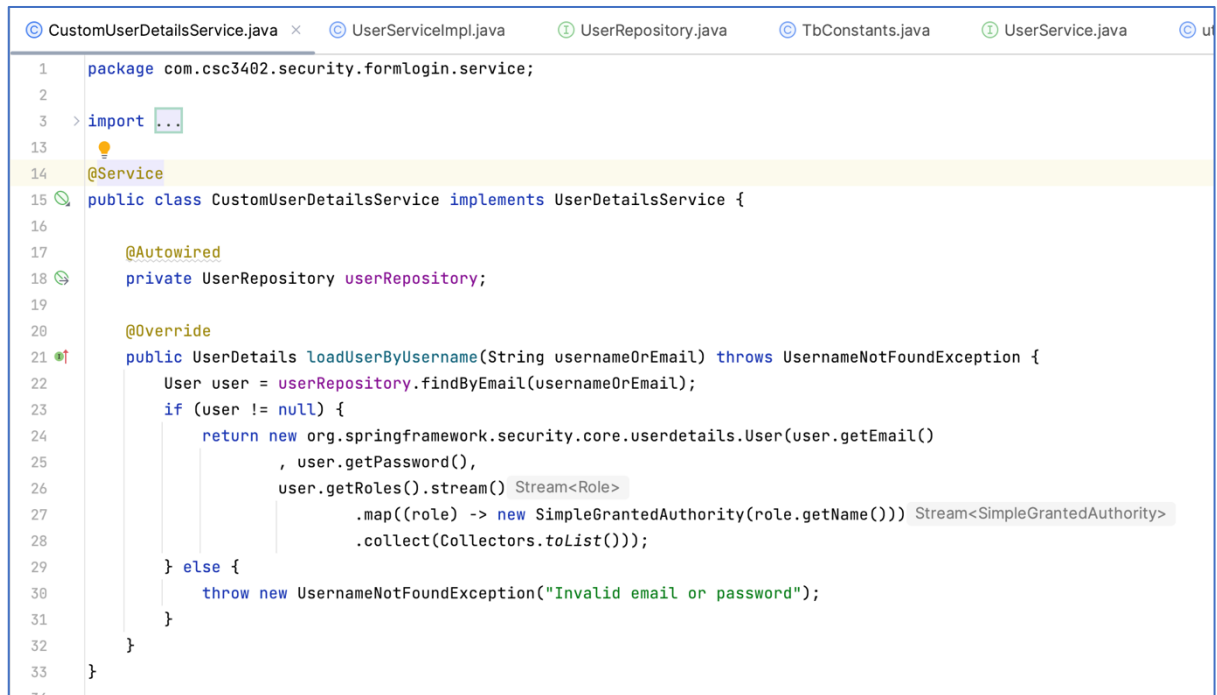
Source path: src/main/java/com/csc3402/security/formlogin/service/UserServiceImpl.java

Below, "CustomUserDetailsService" is a custom implementation of "UserDetailsService" and is used by Spring to get user details from the database during a login flow.

What is the use of UserDetailsService?

The UserDetailsService interface from Spring Security is used to retrieve user-related data. It has one method named loadUserByUsername() which can be overridden to customize the process of finding the user. It is used by the DaoAuthenticationProvider to load details about the user during authentication.

Note: CustomUserDetailsService is annotated with `@Service` annotation.



```
1 package com.csc3402.security.formlogin.service;
2
3 > import ...
13
14 @Service
15 public class CustomUserDetailsService implements UserDetailsService {
16
17     @Autowired
18     private UserRepository userRepository;
19
20     @Override
21     public UserDetails loadUserByUsername(String usernameOrEmail) throws UsernameNotFoundException {
22         User user = userRepository.findByEmail(usernameOrEmail);
23         if (user != null) {
24             return new org.springframework.security.core.userdetails.User(user.getEmail()
25                 , user.getPassword(),
26                 user.getRoles().stream() Stream<Role>
27                     .map((role) -> new SimpleGrantedAuthority(role.getName())) Stream<SimpleGrantedAuthority>
28                     .collect(Collectors.toList()));
29         } else {
30             throw new UsernameNotFoundException("Invalid email or password");
31         }
32     }
33 }
```

Source path: src/main/java/com/csc3402/security/formlogin/service/CustomUserDetailsService.java

Controller Package

This "LoginController" hosts "request-mappings" for "Login" and "Registration" related UI (HTML + Thymeleaf)

```

4  > import ...
16
17  @Controller
18  public class LoginController {
19      @Autowired
20      private UserService userService;
21
22      @RequestMapping("/login")
23      public String loginForm() { return "login"; }
26
27      @GetMapping("/registration")
28      public String registrationForm(Model model) {
29          UserDto user = new UserDto();
30          model.addAttribute("user", user);
31          return "registration";
32      }
33
34      @PostMapping("/registration")
35      public String registration(
36          @Valid @ModelAttribute("user") UserDto userDto,
37          BindingResult result,
38          Model model) {
39          User existingUser = userService.findUserByEmail(userDto.getEmail());
40
41          if (existingUser != null)
42              result.rejectValue("email", null,
43                  "User already registered !!!");
44
45          if (result.hasErrors()) {
46              model.addAttribute("user", userDto);
47              return "/registration";
48          }
49
50          userService.saveUser(userDto);
51          return "redirect:/registration?success";
52      }
53
54      @GetMapping("/logout")
55      public String userLogout() { return "redirect:/login?logout"; }
58  }
59

```

Source path: src/main/java/com/csc3402/security/formlogin/controller/LoginController.java

This "UserController" hosts a single "request-mappings" for "user-details" related UI (HTML + Thymeleaf).

```
UserController.java x LoginController.java CustomUserDetailsService.java
1 package com.csc3402.security.formlogin.controller;
2
3 import org.springframework.stereotype.Controller;
4 import org.springframework.web.bind.annotation.GetMapping;
5 import org.springframework.web.bind.annotation.RequestMapping;
6
7 @Controller
8 @RequestMapping("/user/")
9 public class UserController {
10
11     @GetMapping("/")
12     public String registrationForm() { return "user"; }
13
14 }
15
16
```

Configuration Package (Security Configuration)

The "SecurityFilterChain" defines a filter chain that is capable of being matched against an HttpServletRequest.

As mentioned in **the configuration below**, the urls matching with "/user/**" are accessible by users having either the "USER" or "ADMIN" role, whereas the urls matching with "/admin/**" are only accessible by users having the "ADMIN" role.

On the other hand, urls beginning with "/registration/**" and "/login/**" are accessible by both authenticated and non-authenticated users, or accessible by all.

Create a package called "config" and a java class called "SpringSecurity". Code as follows:-

```
SpringSecurity.java x UserController.java LoginController.java CustomUserDetailsService.java UserServiceImpl.java
1 package com.csc3402.security.formlogin.config;
2
3 > import ...
11
12 @Configuration
13 @EnableWebSecurity
14 public class SpringSecurity {
15
16     @Bean
17     public static PasswordEncoder passwordEncoder() { return new BCryptPasswordEncoder(); }
18
19
20
21     @Bean
22     public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
23         http
24             .authorizeHttpRequests((requests) -> requests
25                 .requestMatchers(AntPathRequestMatcher.antMatcher("/h2-console/**")).permitAll()
26                 .requestMatchers("/registration/**").permitAll()
27                 .requestMatchers("/login/**").permitAll()
28                 .requestMatchers("/user/**").hasAnyRole("USER", "ADMIN")
29                 .requestMatchers("/admin/**").hasAnyAuthority("ADMIN")
30                 .anyRequest().authenticated()
31             )
32             .formLogin((form) -> form
33                 .loginPage("/login")
34                 .loginProcessingUrl("/login")
35                 .defaultSuccessUrl("/user/")
36                 .permitAll()
37             )
38             .logout((logout) -> logout.permitAll())
39             .exceptionHandling((exceptionHandling) -> exceptionHandling.accessDeniedPage("/access-denied"))
40             .csrf((csrf) -> csrf.ignoringRequestMatchers(AntPathRequestMatcher.antMatcher("/h2-console/**")))
41             .headers((headers) -> headers.disable());
42         return http.build();
43     }
44 }
```

Source path: src/main/java/com/csc3402/security/formlogin/config/SpringSecurity.java

Application

This is the simple "spring boot" "main" class; all the application execution starts from here. The only interesting thing to note is the exclusion of "SecurityAutoConfiguration.class" and "ManagementWebSecurityAutoConfiguration.class". This is required to disable security auto-configuration so that we can have our own settings, like a custom login form.

```
FormLoginApplication.java x pom.xml (form-login) UserController.java LoginController.java CustomUserDetailsService.java
1 package com.csc3402.security.formlogin;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.actuate.autoconfigure.security.servlet.ManagementWebSecurityAutoConfiguration;
5 import org.springframework.boot.autoconfigure.SpringBootApplication;
6 import org.springframework.boot.autoconfigure.security.servlet.SecurityAutoConfiguration;
7
8 @SpringBootApplication(exclude = {SecurityAutoConfiguration.class, ManagementWebSecurityAutoConfiguration.class})
9 public class FormLoginApplication {
10
11     public static void main(String[] args) { SpringApplication.run(FormLoginApplication.class, args); }
12
13
14
15 }
16 }
```

Source path: src/main/java/com/csc3402/security/formlogin/FormLoginApplication.java

HTML

The "registration.html" is a simple ".html" file with "thymeleaf" template capabilities. This page contains a server-side validation-enabled "registration" form.

HTML code: registration.html

```
<!doctype html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="utf-8">
  <meta content="width=device-width, initial-scale=1" name="viewport">
  <link crossorigin="anonymous"
href="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/css/bootstrap.min.css"
  integrity="sha384-EVSTQN3/azprG1Anm3QDgpJLIm9Nao0Yz1ztcQTwFspd3yD65VohhpuuCOmLASjC"
  rel="stylesheet">

  <title>Registration</title>
</head>
<body>
<div class="container">
  <div class="row mt-5">
    <div class="col-6 mx-auto">
      <form
        method="post"
        role="form"
        th:action="@{/registration}"
        th:object="${user}"
      >
        <h1>Registration Form</h1>
        <div th:if="${param.success}">
          <div class="alert alert-info">
            Registration successful !!!
          </div>
        </div>
        <div class="mb-3">
          <label class="form-label" for="name">Name</label>
          <input class="form-control" id="name" name="name" type="text">
          <p class="text-danger" th:errors="*{name}"
            th:if="${#fields.hasErrors('name')}"></p>
        </div>
        <div class="mb-3">
          <label class="form-label" for="email">Email</label>
          <input class="form-control" id="email" name="email" type="email">
          <p class="text-danger" th:errors="*{email}"
            th:if="${#fields.hasErrors('email')}"></p>
        </div>
        <div class="mb-3">
          <label class="col-sm-2 col-form-label" for="password">Password</label>
          <input class="form-control" id="password" name="password" type="password">
          <p class="text-danger" th:errors="*{password}"
            th:if="${#fields.hasErrors('password')}"></p>
        </div>
        <div class="mb-3">
          <button class="btn btn-primary mb-3" type="submit">Register</button>
          or <a class="link-primary" th:href="@{/login}">Login</a>
        </div>
      </form>
    </div>
  </div>
</div>

<script crossorigin="anonymous"
  integrity="sha384-MrcW6ZMFYlzcLA8Nl+NtUVF0sA7MsXsP1UyJoMp4YLEuNSfAP+JcXn/tWtIaxVXM"
src="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/js/bootstrap.bundle.min.js"></script>
>

</body>
</html>
```

Source path: src/main/resources/templates/registration.html

By default spring security accepts login request at "/login" and credentials in the form of "username" and "password". Custom design user login form.

HTML code:login.html

```
<!doctype html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="utf-8">
  <meta content="width=device-width, initial-scale=1" name="viewport">
  <link crossorigin="anonymous"
href="https://cdn.jsdelivrivr.net/npm/bootstrap@5.0.2/dist/css/bootstrap.min.css"
  integrity="sha384-EVSTQN3/azprG1Anm3QDgpJLIm9Nao0Yz1ztcQTwFspd3yD65VohhpuuCOmLASjC"
rel="stylesheet">

  <title>Login</title>
</head>
<body>
<div class="container">
  <div class="row mt-5">
    <div class="col-6 mx-auto">
      <h1>Login Form</h1>
      <div th:if="{param.error}">
        <div class="alert alert-danger">Invalid Email or Password !!!</div>
      </div>
      <div th:if="{param.logout}">
        <div class="alert alert-success">You have been logged out !!!</div>
      </div>
      <form
        class="form-horizontal"
        method="post"
        role="form"
        th:action="@{/login}"
      >
        <div class="mb-3">
          <label class="form-label" for="password">Email</label>
          <input class="form-control" id="username" name="username" placeholder="Enter
email address"
            type="email">
        </div>
        <div class="mb-3">
          <label class="col-sm-2 col-form-label" for="password">Password</label>
          <input class="form-control" id="password" name="password" placeholder="Enter
password"
            type="password">
        </div>
        <div class="mb-3">
          <button class="btn btn-primary mb-3" type="submit">Login</button>
          or <a class="link-primary" th:href="@{/registration}">Register</a>
        </div>
      </form>
    </div>
  </div>
</div>

<script crossorigin="anonymous"
  integrity="sha384-MrcW6ZMFYlzcLA8Nl+NtUVF0sA7MsXsPlUyJoMp4YLEuNSfAP+JcXn/tWtIaxVXM"
src="https://cdn.jsdelivrivr.net/npm/bootstrap@5.0.2/dist/js/bootstrap.bundle.min.js"></script>
>

</body>
</html>
```

Source path: src/main/resources/templates/login.html

The "thymeleaf" template engine enables us to use "sec:authentication" tags to display security-related information in HTML.

HTML code: user.html

```
<!doctype html>
<html lang="en" xmlns:sec="http://www.thymeleaf.org/extras/spring-security"
      xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="utf-8">
  <meta content="width=device-width, initial-scale=1" name="viewport">
  <link crossorigin="anonymous"
href="https://cdn.jsdelivrivr.net/npm/bootstrap@5.0.2/dist/css/bootstrap.min.css"
      integrity="sha384-EVSTQN3/azprG1Anm3QDgpJLIm9Nao0Yz1ztcQTWfSpd3yD65VohhpuuCOmLASjC"
rel="stylesheet">

  <title>User</title>
</head>
<body>
<div class="container">
  <div class="row mt-5">
    <div class="col-6 mx-auto">
      <h1>User Details</h1>
      <div class="mt-2">Username: <span sec:authentication="principal.username"/></div>
      <div class="mt-2">Roles: <span sec:authentication="principal.authorities"/></div>

      <div class="mt-5"><a class="link-primary" th:href="@{/logout}">Logout</a></div>
    </div>
  </div>
</div>

<script crossorigin="anonymous"
      integrity="sha384-MrcW6ZMFYlzcLA8Nl+NtUVF0sA7MsXsP1UyJoMp4YLEuNSfAP+JcXn/tWtIaxVXM"

src="https://cdn.jsdelivrivr.net/npm/bootstrap@5.0.2/dist/js/bootstrap.bundle.min.js"></script
>

</body>
</html>
```

Source path: src/main/resources/templates/user.html

Application Properties

This "application.properties" file contains "db credentials," "hibernate" settings configurations.

```
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
# In order to get import_backup.sql populate data into H2 table
spring.sql.init.mode=always
# For Spring Boot 2.4+ add spring.jpa.defer-datasource-initialization=true in application.properties
spring.jpa.defer-datasource-initialization=true
# H2 Database Setting
spring.h2.console.enabled=true
###
# Hibernate Settings
# ddl.auto = none | validate | update | create | create-drop
spring.jpa.hibernate.ddl-auto = create-drop
spring.jpa.properties.hibernate.show_sql=true
spring.jpa.properties.hibernate.use_sql_comments=false
spring.jpa.properties.hibernate.format_sql=true
```

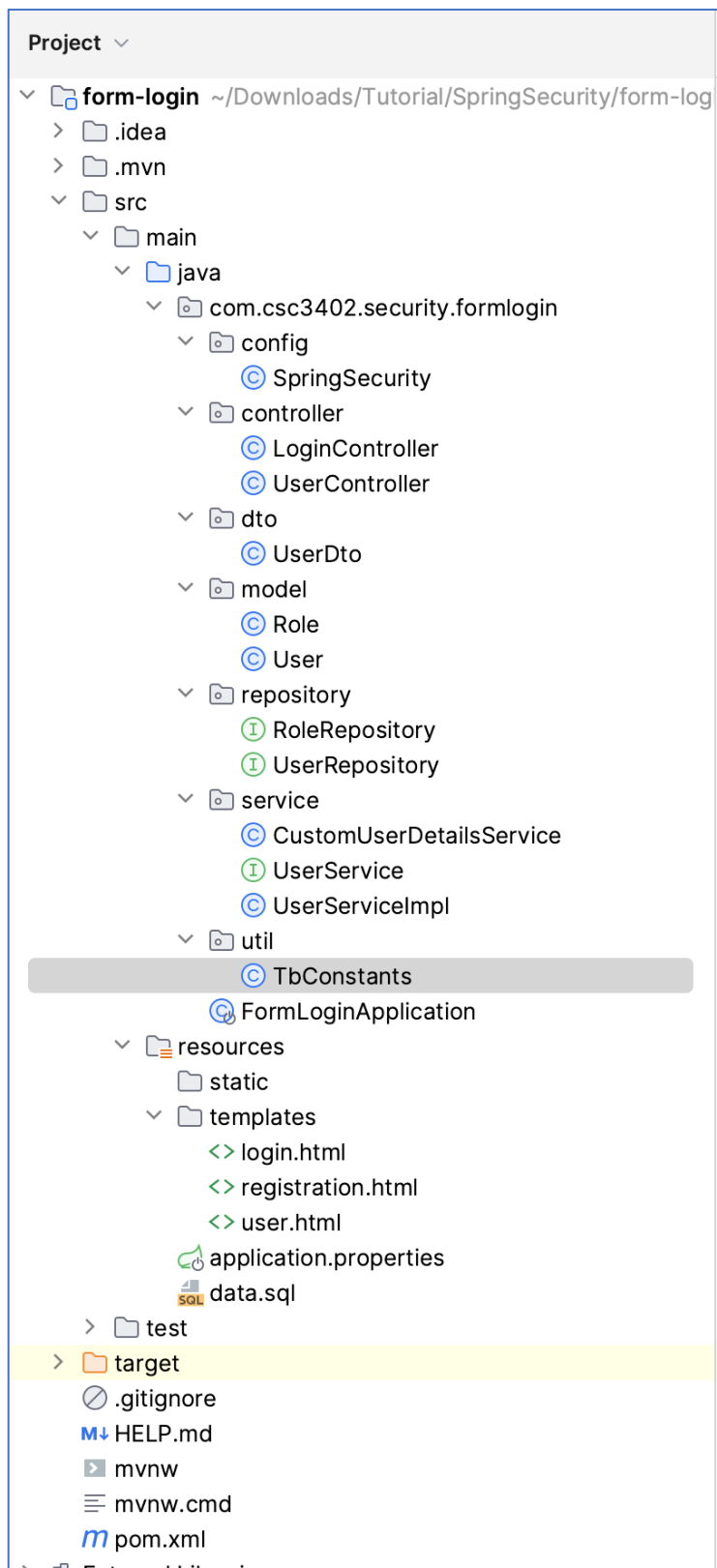
File name: data.sql

```
INSERT INTO users(email, name, password) VALUES
('admin@gmail.com','admin','$2a$12$JGriZzgFwZNEeulzFcocjug9wb0/G0EJ1nco27FZoCvVLmfpfiiWe');
INSERT INTO users(email, name, password) VALUES
('test@gmail.com','John','$2a$12$F2QRPx07EsQTswGmyxB4sOw7RLCnIDqux/LWhO5vfegs42OS2LE.C');
INSERT INTO users(email, name, password) VALUES
('abc@gmail.com','Alex','$2a$12$SaRRjmnRA5MkVg3M.xSc1G.jQkkvfx73WJhlvl.77IMkJyZpTT5dvC');
INSERT INTO users(email, name, password) VALUES
('def@gmail.com','Boss','$2a$12$MBelKc4HGARn/pOnBl1HSugtEBvL5EwRvQ4EzqAykvt4hUogKI/Zy');

INSERT INTO roles(name) VALUES ('ROLE_ADMIN');
INSERT INTO roles(name) VALUES ('ROLE_USER');

INSERT INTO users_roles(role_id, user_id) VALUES (1,1);
INSERT INTO users_roles(role_id, user_id) VALUES (2,2);
INSERT INTO users_roles(role_id, user_id) VALUES (2,3);
INSERT INTO users_roles(role_id, user_id) VALUES (2,4);
INSERT INTO users_roles(role_id, user_id) VALUES (1,4);
```


Program Structure

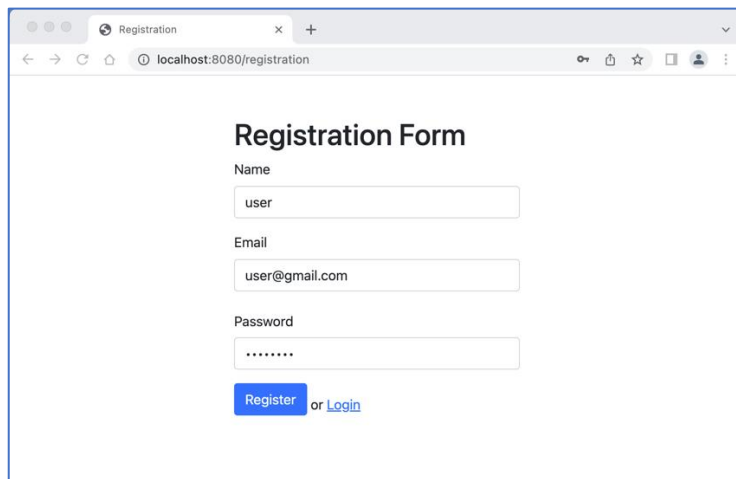


Testing

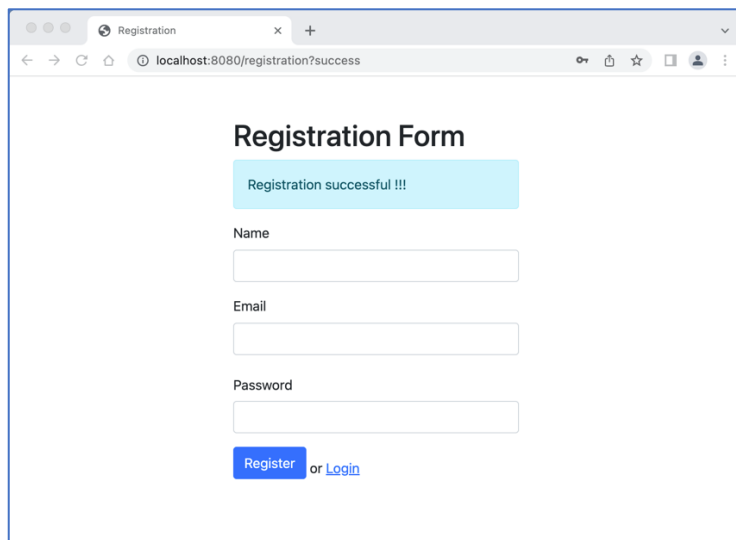
1) Registration

Let's run the application and start testing the functionality by registering a new user at : <http://localhost:8080/registration>.

Notes: Currently all uploaded users (data.sql) are assigned password as “password”, for testing purpose.



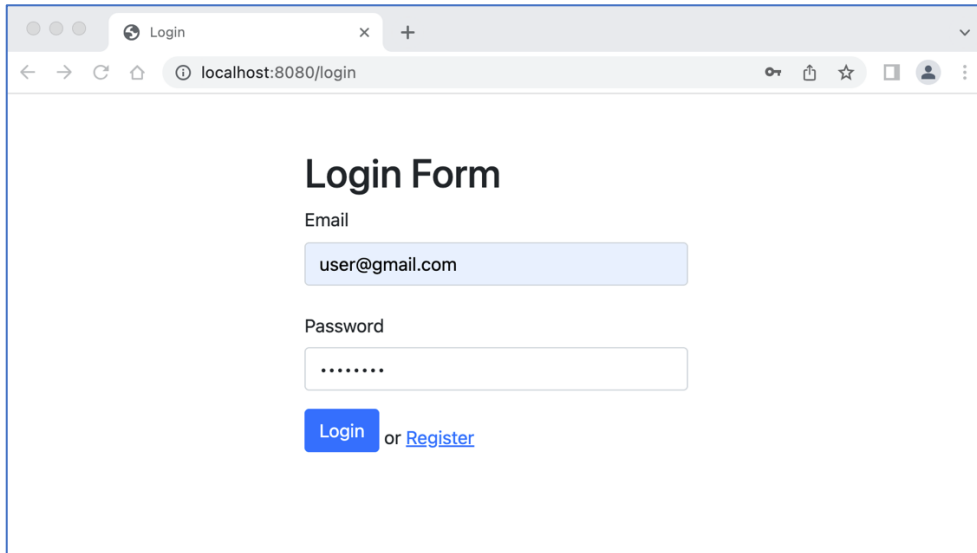
A screenshot of a web browser showing the 'Registration Form' page. The browser's address bar displays 'localhost:8080/registration'. The form contains three input fields: 'Name' with the value 'user', 'Email' with the value 'user@gmail.com', and 'Password' with masked characters '.....'. Below the fields is a blue 'Register' button followed by the text 'or [Login](#)'.



A screenshot of the same 'Registration Form' page, but the browser's address bar now shows 'localhost:8080/registration?success'. A light blue success message 'Registration successful !!!' is displayed at the top of the form area. The input fields for 'Name', 'Email', and 'Password' are currently empty. The 'Register' button and 'or [Login](#)' link remain at the bottom.

2) Login

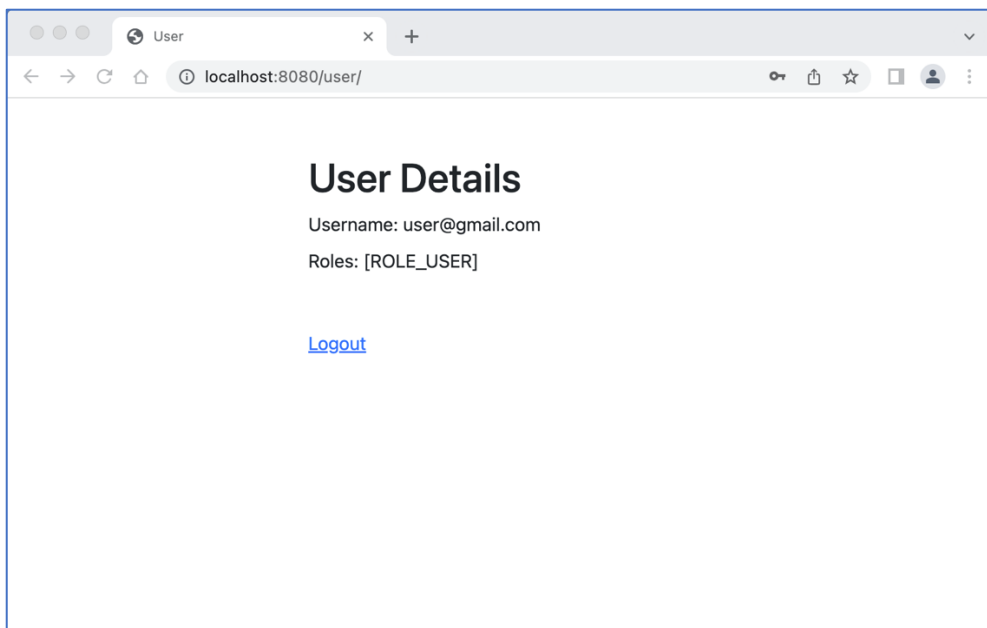
Now that a new user has been successfully registered, we can use the credentials to login at: <http://localhost:8080/login>.



The screenshot shows a web browser window with the title "Login". The address bar displays "localhost:8080/login". The page content includes a heading "Login Form", an "Email" label, a text input field containing "user@gmail.com", a "Password" label, a password input field with masked characters ".....", and a blue "Login" button followed by the text "or [Register](#)".

3) Details

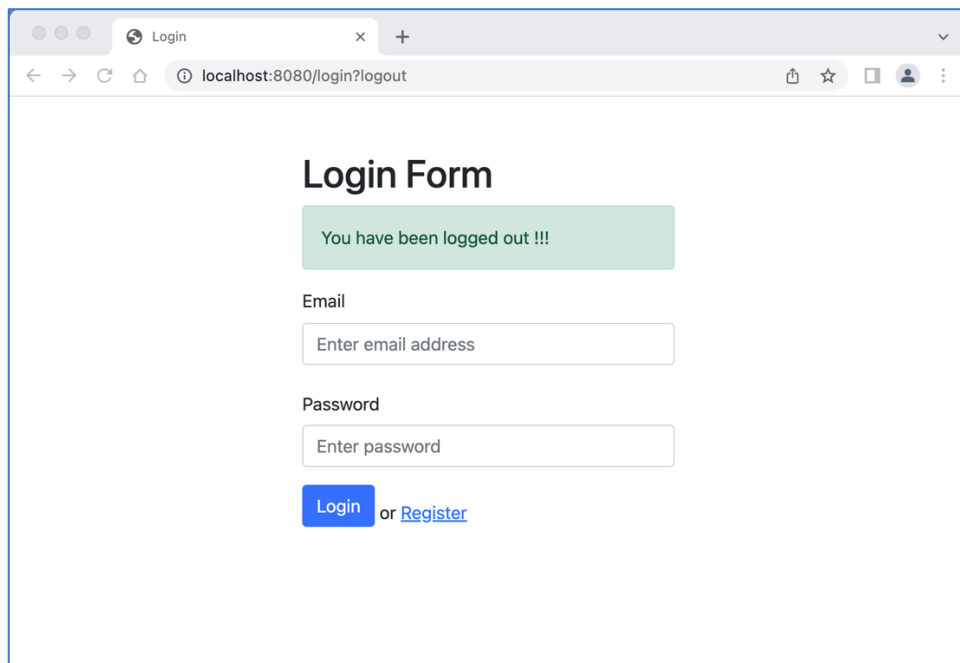
The user is now "logged-in" to the system with "role_user" and hence URLs accessible by "role_user" can be viewed: <http://localhost:8080/user/>



The screenshot shows a web browser window with the title "User". The address bar displays "localhost:8080/user/". The page content includes a heading "User Details", the text "Username: user@gmail.com", the text "Roles: [ROLE_USER]", and a blue [Logout](#) link.

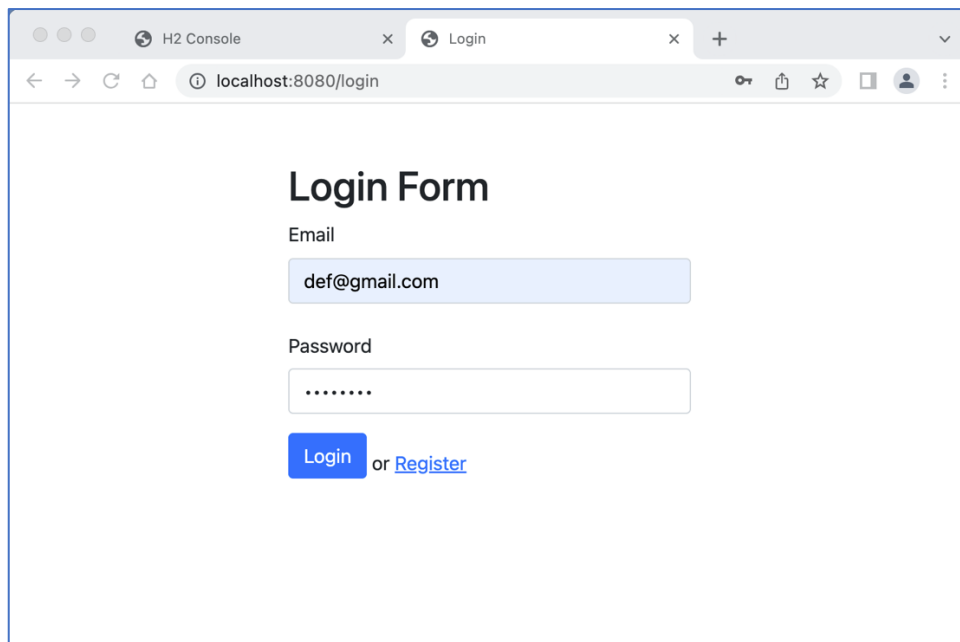
3) Logout

In order to log-out from the system, we can send a get call to <http://localhost:8080/logout> or click on **Logout** link , please refer to the diagram above.

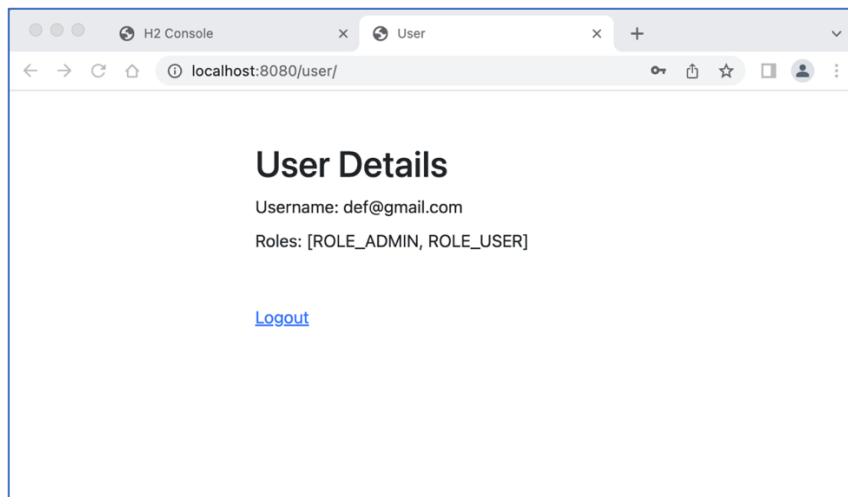


A screenshot of a web browser window showing the 'Login Form' page. The browser's address bar displays 'localhost:8080/login?logout'. A green message box at the top of the form states 'You have been logged out !!!'. Below this, there are input fields for 'Email' (with placeholder text 'Enter email address') and 'Password' (with placeholder text 'Enter password'). At the bottom of the form, there is a blue 'Login' button followed by the text 'or [Register](#)'.

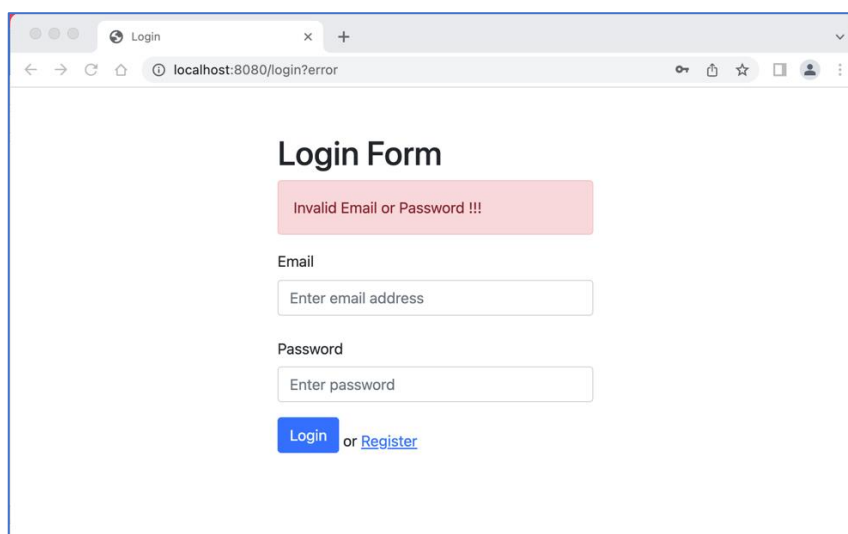
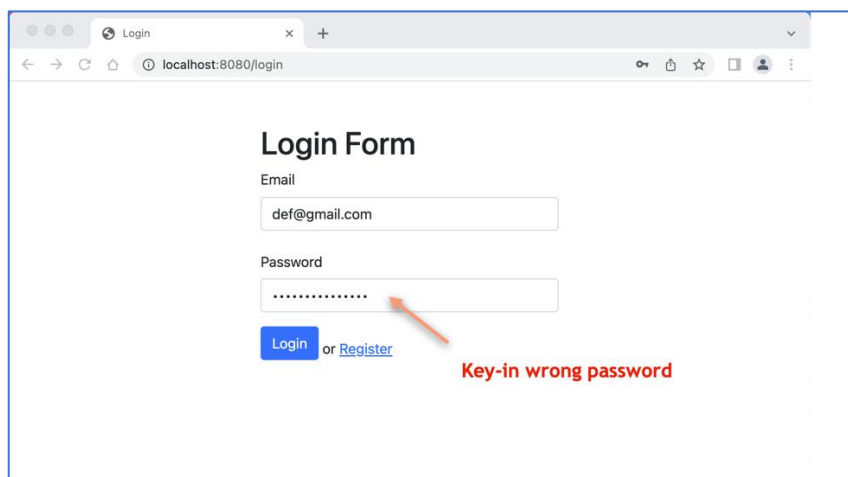
4) Login user with two roles – ADMIN & USER



A screenshot of a web browser window showing the 'Login Form' page. The browser has two tabs: 'H2 Console' and 'Login'. The address bar shows 'localhost:8080/login'. The 'Email' input field is pre-filled with 'def@gmail.com'. The 'Password' input field contains masked characters represented by dots. At the bottom, there is a blue 'Login' button followed by the text 'or [Register](#)'.



5) Wrong password



H2 Data

The screenshot shows the H2 Console interface. The left sidebar displays the database schema: jdbc:h2:mem:testdb, ROLES, USERS, USERS_ROLES, INFORMATION_SCHEMA, and Users. The main area shows the SQL statement: `SELECT * FROM USERS`. The results are displayed in a table with 5 rows and 4 columns: ID, EMAIL, NAME, and PASSWORD.

ID	EMAIL	NAME	PASSWORD
1	admin@gmail.com	admin	\$2a\$12\$JGrIZgFwZNEultzFcojug9wb0/G0EJ1nco27Fz0CvVlmpfllWe
2	test@gmail.com	John	\$2a\$12\$F2QRPx07EsQTawGmyx84sOw7RLCnLDqux/LWhO5fegs42OS2LE.C
3	abc@gmail.com	Alex	\$2a\$12\$aRRjmnRASmKvG3M.xSc1G.J0kkvfx73W.Jhlv1.77IMk.JyZpTT5dvC
4	def@gmail.com	Boss	\$2a\$12\$MBelKc4HGAtn/pOnBI1HSugtEBvL5EwRvQ4EzqAykvt4hUogKiZy
5	user@gmail.com	user	\$2a\$10\$quEopLqJysNZOxHdJ1qsOpIDF7OKBchjw8gJ6HkSMamKHPj7ip12

(5 rows, 1 ms)

The screenshot shows the H2 Console interface. The left sidebar displays the database schema: jdbc:h2:mem:testdb, ROLES, USERS, USERS_ROLES, INFORMATION_SCHEMA, and Users. The main area shows the SQL statement: `SELECT * FROM USERS_ROLES`. The results are displayed in a table with 6 rows and 2 columns: ROLE_ID and USER_ID.

ROLE_ID	USER_ID
1	1
2	2
2	3
2	4
1	4
2	5

(6 rows, 0 ms)

Note: USER_ID : 4 having two roles. ROLE_ID: 2 and 1

The screenshot shows the H2 Console interface. The left sidebar displays the database schema: jdbc:h2:mem:testdb, ROLES, USERS, USERS_ROLES, INFORMATION_SCHEMA, and Users. The main area shows the SQL statement: `SELECT * FROM ROLES`. The results are displayed in a table with 2 rows and 2 columns: ID and NAME.

ID	NAME
1	ROLE_ADMIN
2	ROLE_USER

(2 rows, 0 ms)